

Web Technology

Class Lecture
+
Lab Session
10 Hours

Unit- CSS

Unit III: Cascading Style Sheets

3.1. Introduction; Cascading Style Sheets (CSS); CSS Syntax

3.2. Inserting CSS: Inline, Internal, External 3.3. CSS ID and Class Selectors

3.4. Colors; Backgrounds; Borders; Text; Font; List; Table

3.5. CSS Box Model; Normal Flow Box Layout: Basic Box Layout

3.6. Display Property, Padding, Margin; Positioning: Relative, Float, Absolute

3.7. CSS3 Borders, Box Shadows, Text Effects and shadow

3.8. Basics of Responsive Web Designs; Media Queries (Media Types, Viewport)

3.9. Introduction to Bootstrap (Basic concepts and installation) Class Lecture + Lab Session 8 Hours Unit IV:

Client Side Scripting with JavaScript

What is CSS

- CSS stands for Cascading Style Sheets.
- It is a style sheet language which is used to describe the look and formatting of a document written in markup language. It provides an additional feature to HTML.
- It is generally used with HTML to change the style of web pages and user interfaces.
- It can also be used with any kind of XML documents including plain XML, SVG and XUL.
- CSS is used along with HTML and JavaScript in most websites to create user interfaces for web applications and user interfaces for many mobile applications.

What does CSS do ?

- You can add new looks to your old HTML documents.
- You can completely change the look of your website with only a few changes in CSS code.

Why use CSS

These are the three major benefits of CSS:

1) Solves a big problem

Before CSS, tags like font, color, background style, element alignments, border and size had to be repeated on every web page. This was a very long process. For example: If you are developing a large website where fonts and color information are added on every single page, it will be become a long and expensive process. CSS was created to solve this problem. It was a W3C recommendation.

2) Saves a lot of time

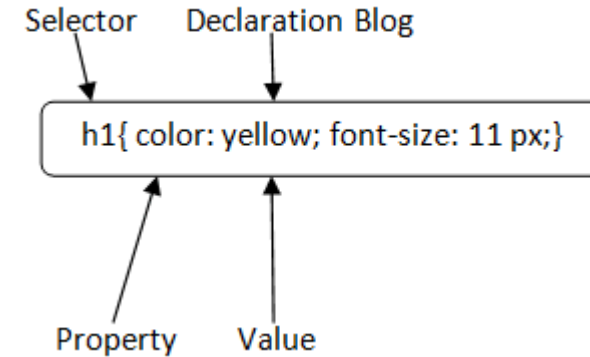
CSS style definitions are saved in external CSS files so it is possible to change the entire website by changing just one file.

3) Provide more attributes

CSS provides more detailed attributes than plain HTML to define the look and feel of the website.

CSS Syntax

A CSS rule set contains a selector and a declaration block.



Selector: Selector indicates the HTML element you want to style. It could be any tag like `<h1>`, `<title>` etc.

Declaration Block: The declaration block can contain one or more declarations separated by a semicolon. For the above example, there are two declarations:

`color: yellow;`
`font-size: 11 px;`

Each declaration contains a property name and value, separated by a colon.

Property: A Property is a type of attribute of HTML element. It could be color, border etc.

Value: Values are assigned to CSS properties. In the above example, value "yellow" is assigned to color property.

Selector{Property1: value1; Property2: value2;;}

CSS Selector

CSS selectors are used to select the content you want to style. Selectors are the part of CSS rule set. CSS selectors select HTML elements according to its id, class, type, attribute etc.

There are several different types of selectors in CSS.

1. CSS Element Selector
2. CSS Id Selector
3. CSS Class Selector
4. CSS Universal Selector
5. CSS Group Selector

Some one can say like

A CSS selector selects the HTML element(s) you want to style.

CSS Selectors

CSS selectors are used to "find" (or select) the HTML elements you want to style.

We can divide CSS selectors into five categories:

1. Simple selectors (select elements based on name, id, class)
2. Combinator selectors (select elements based on a specific relationship between them)
3. Pseudo-class selectors (select elements based on a certain state)
4. Pseudo-elements selectors (select and style a part of an element)
5. Attribute selectors (select elements based on an attribute or attribute value)

The CSS element Selector

The element selector selects HTML elements based on the element name.

Example

Here, all <p> elements on the page will be center-aligned, with a red text color:

```
p {  
  text-align: center;  
  color: red;  
}
```

The CSS class Selector

The class selector selects HTML elements with a specific class attribute.

To select elements with a specific class, write a period (.) character, followed by the class name.

Example

In this example all HTML elements with class="center" will be red and center-aligned:

```
.center {  
  text-align: center;  
  color: red;  
}
```

The CSS id Selector

The id selector uses the id attribute of an HTML element to select a specific element.

The id of an element is unique within a page, so the id selector is used to select one unique element!

To select an element with a specific id, write a hash (#) character, followed by the id of the element.

Example

The CSS rule below will be applied to the HTML element with id="para1":

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

You can also specify that only specific HTML elements should be affected by a class.

The element selector selects the HTML element by name.

```
<!DOCTYPE html>
<html>
<head>
<style>
p{
  text-align: center;
  color: blue;
}
</style>
</head>
<body>
<p>This style will be applied on every paragraph.</p>
<p id="para1">Me too!</p>
<p>And me!</p>
</body>
</html>
```

CSS Id Selector

```
<!DOCTYPE html>
<html>
<head>
<style>
#para1 {
  text-align: center;
  color: blue;
}
</style>
</head>
<body>
<p id="para1">Hello Javatpoint.com</p>
<p>This paragraph will not be affected.</p>
</body>
</html>
```

CSS Class Selector

```
<!DOCTYPE html>
<html>
<head>
<style>
.center {
    text-align: center;
    color: blue;
}
</style>
</head>
<body>
<h1 class="center">This heading is blue and center-aligned.</h1>
<p class="center">This paragraph is blue and center-aligned.</p>
</body>
</html>
```

CSS Class Selector for specific element

If you want to specify that only one specific HTML element should be affected then you should use the element name with class selector.

```
<!DOCTYPE html>
<html>
<head>
<style>
p.center {
    text-align: center;
    color: blue;
}
</style>
</head>
<body>
<h1 class="center">This heading is not affected</h1>
<p class="center">This paragraph is blue and center-
aligned.</p>
</body>
</html>
```

CSS Universal Selector

The universal selector is used as a wildcard character. It selects all the elements on the pages.

```
<!DOCTYPE html>
<html>
<head>
<style>
* {
  color: green;
  font-size: 20px;
}
</style>
</head>
<body>
<h2>This is heading</h2>
<p>This style will be applied on every paragraph.</p>
<p id="para1">Me too!</p>
<p>And me!</p>
</body>
</html>
```

CSS Group Selector

The grouping selector is used to select all the elements with the same style definitions.

Grouping selector is used to minimize the code. Commas are used to separate each selector in grouping.

Let's see the CSS code without group selector.

```
h1 {
  text-align: center;
  color: blue;
}
h2 {
  text-align: center;
  color: blue;
}
p {
  text-align: center;
  color: blue;
}
```

As you can see, you need to define CSS properties for all the elements. It can be grouped in following ways:


```
<!DOCTYPE html>
<html>
<head>
<style>
h1, h2, p {
    text-align: center;
    color: blue;
}
</style>
</head>
<body>
<h1>Hello Javatpoint.com</h1>
<h2>Hello Javatpoint.com (In smaller font)</h2>
<p>This is a paragraph.</p>
</body>
</html>
```

❑ How to add CSS

CSS is added to HTML pages to format the document according to information in the style sheet. There are three ways to insert CSS in HTML documents.

1. Inline CSS
2. Internal CSS
3. External CSS

1) Inline CSS

- Inline CSS is used to apply CSS on a single line or element
- We can apply CSS in a single element by inline CSS technique.
- The inline CSS is also a method to insert style sheets in HTML document. This method mitigates some advantages of style sheets so it is advised to use this method sparingly.
- If you want to use inline CSS, you should use the style attribute to the relevant tag.

.
<htmltag style="cssproperty1:value; cssproperty2:value;" > </htmltag>

For example:

<h2 style="color:red;margin-left:40px;">Inline CSS is applied on this heading.</h2>
<p>This paragraph is not affected.</p>

Disadvantages of Inline CSS

- You cannot use quotations within inline CSS. If you use quotations the browser will interpret this as an end of your style value.
- These styles cannot be reused anywhere else.
- These styles are tough to be edited because they are not stored at a single place.
- It is not possible to style pseudo-codes and pseudo-classes with inline CSS.
- Inline CSS does not provide browser cache advantages.

2) Internal CSS

Internal CSS is used to apply CSS on a single document or page. It can affect all the elements of the page. It is written inside the style tag within head section of html.

For example:

```
<style>
p{color:blue}
</style>
```

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
    background-color: linen;
}
h1 {
    color: Red;
    margin-left: 80px;
}
</style>
</head>
<body>
<h1>The internal style sheet is applied on this heading.</h1>
<p>This paragraph will not be affected.</p>
</body>
</html>
```

3. External CSS

- The external style sheet is generally used when you want to make changes on multiple pages. It is ideal for this condition because it facilitates you to change the look of the entire web site by changing just one file.
- It uses the <link> tag on every pages and the <link> tag should be put inside the head section.

Example:

1.<head>

2.<link rel="stylesheet" type="text/css" href="mystyle.css">

3.</head>

The external style sheet may be written in any text editor but must be saved with a .css extension. This file should not contain HTML elements.

Let's take an example of a style sheet file named "mystyle.css".

File: mystyle.css

```
body {  
    background-color: lightblue;  
}  
h1 {  
    color: navy;  
    margin-left: 20px;  
}
```

Note: You should not use a space between the property value and the unit. For example: It should be margin-left:20px not margin-left:20 px.

```
<!DOCTYPE html>  
<html>  
<head>  
<link rel="stylesheet" href="mystyle.css">  
</head>  
<body>  
  
<h1>This is a heading</h1>  
<p>This is a paragraph.</p>  
  
</body>  
</html>
```

CSS Comments

Comments are used to explain the code, and may help when you edit the source code at a later date.

Comments are ignored by browsers.

A CSS comment is placed inside the <style> element, and starts with /* and ends with */:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
/* This is a single-line comment */  
p {  
    color: red;  
}  
</style>  
</head>  
<body>  
  
<p>Hello World!</p>  
<p>This paragraph is styled with CSS.</p>  
<p>CSS comments are not shown in the output.</p>  
  
</body>  
</html>
```

CSS Colors

Colors are specified using predefined color names, or RGB, HEX, HSL, RGBA, HSLA values.



CSS/HTML support 140 standard color names.

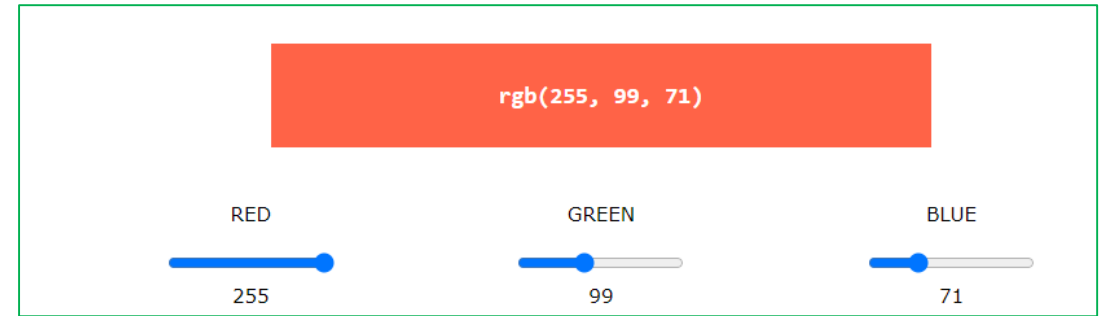
```
<!DOCTYPE html>
<html>
<body>

<h1 style="background-color:Tomato;">Tomato</h1>
<h1 style="background-color:Orange;">Orange</h1>
<h1 style="background-color:DodgerBlue;">DodgerBlue</h1>
<h1 style="background-color:MediumSeaGreen;">MediumSeaGreen</h1>
<h1 style="background-color:Gray;">Gray</h1>
<h1 style="background-color:SlateBlue;">SlateBlue</h1>
<h1 style="background-color:Violet;">Violet</h1>
<h1 style="background-color:LightGray;">LightGray</h1>

</body>
</html>
```

CSS RGB Colors

- An RGB color value represents RED, GREEN, and BLUE light sources.
- RGB Value
- In CSS, a color can be specified as an RGB value, using this formula:
 - `rgb(red, green, blue)`
 - Each parameter (red, green, and blue) defines the intensity of the color between 0 and 255.
 - For example, `rgb(255, 0, 0)` is displayed as red, because red is set to its highest value (255) and the others are set to 0.
 - To display black, set all color parameters to 0, like this: `rgb(0, 0, 0)`.
 - To display white, set all color parameters to 255, like this: `rgb(255, 255, 255)`.
 - Experiment by mixing the RGB values below:



Example



```

<!DOCTYPE html>
<html>
<body>

<h1>Specify colors using RGB values</h1>

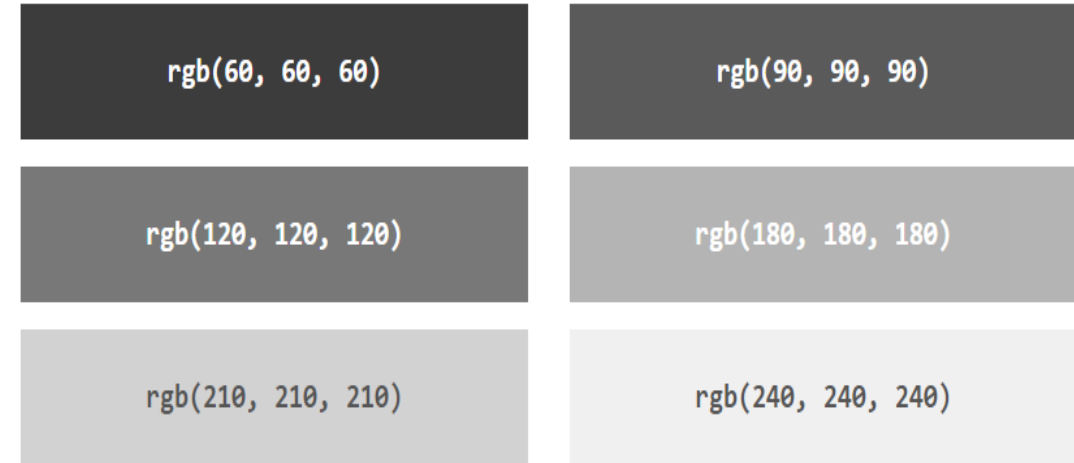
<h2 style="background-color:rgb(255, 0, 0);">rgb(255, 0, 0)</h2>
<h2 style="background-color:rgb(0, 0, 255);">rgb(0, 0, 255)</h2>
<h2 style="background-color:rgb(60, 179, 113);">rgb(60, 179,
113)</h2>
<h2 style="background-color:rgb(238, 130, 238);">rgb(238, 130,
238)</h2>
<h2 style="background-color:rgb(255, 165, 0);">rgb(255, 165, 0)</h2>
<h2 style="background-color:rgb(106, 90, 205);">rgb(106, 90,
205)</h2>

</body>
</html>

```

Shades of gray are often defined using equal values for all the 3 light sources:

Example



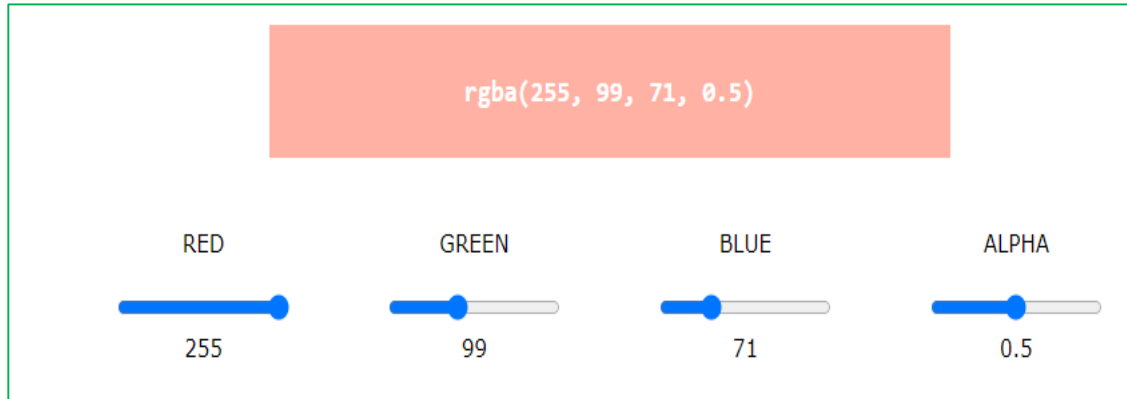
```

<h2 style="background-color:rgb(60, 60, 60);">rgb(60, 60, 60)</h2>
<h2 style="background-color:rgb(90, 90, 90);">rgb(90, 90, 90)</h2>
<h2 style="background-color:rgb(120, 120, 120);">rgb(120, 120, 120)</h2>
<h2 style="background-color:rgb(180, 180, 180);">rgb(180, 180, 180)</h2>
<h2 style="background-color:rgb(210, 210, 210);">rgb(210, 210, 210)</h2>
<h2 style="background-color:rgb(240, 240, 240);">rgb(240, 240, 240)</h2>

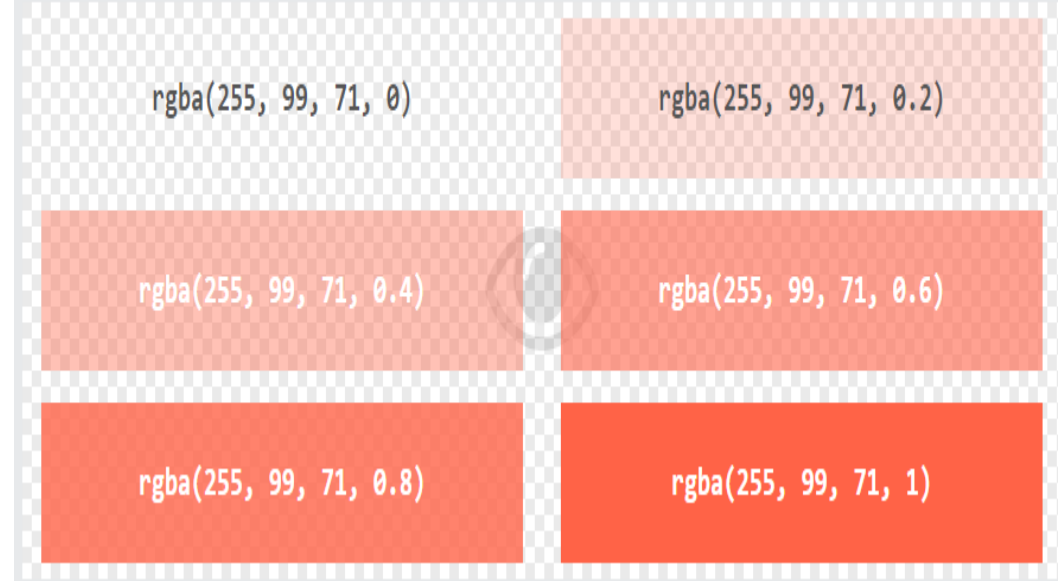
```

RGBA Value

- RGBA color values are an extension of RGB color values with an alpha channel - which specifies the opacity for a color.
- An RGBA color value is specified with:
- **rgba(red, green, blue, alpha)**
- The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (not transparent at all):
- Experiment by mixing the RGBA values below:



Example



```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>Make transparent colors with RGBA</h1>
```

```
<h2 style="background-color:rgba(255, 99, 71, 0);">rgba(255, 99, 71, 0)</h2>
```

```
<h2 style="background-color:rgba(255, 99, 71, 0.2);">rgba(255, 99, 71, 0.2)</h2>
```

```
<h2 style="background-color:rgba(255, 99, 71, 0.4);">rgba(255, 99, 71, 0.4)</h2>
```

```
<h2 style="background-color:rgba(255, 99, 71, 0.6);">rgba(255, 99, 71, 0.6)</h2>
```

```
<h2 style="background-color:rgba(255, 99, 71, 0.8);">rgba(255, 99, 71, 0.8)</h2>
```

```
<h2 style="background-color:rgba(255, 99, 71, 1);">rgba(255, 99, 71, 1)</h2>
```

```
</body>
```

```
</html>
```


CSS HEX Colors

A hexadecimal color is specified with: #RRGGBB, where the RR (red), GG (green) and BB (blue) hexadecimal integers specify the components of the color.

- In CSS, a color can be specified using a hexadecimal value in the form:
- #rrggbb
- Where rr (red), gg (green) and bb (blue) are hexadecimal values between 00 and ff (same as decimal 0-255).
- For example, #ff0000 is displayed as red, because red is set to its highest value (ff) and the others are set to the lowest value (00).
- To display black, set all values to 00, like this: #000000.
- To display white, set all values to ff, like this: #ffffff.
- Experiment by mixing the HEX values below:

```
<!DOCTYPE html>
<html>
<body>

<h1>Specify colors using HEX values</h1>

<h2 style="background-color:#ff0000;">#ff0000</h2>
<h2 style="background-color:#0000ff;">#0000ff</h2>
<h2 style="background-color:#3cb371;">#3cb371</h2>
<h2 style="background-color:#ee82ee;">#ee82ee</h2>
<h2 style="background-color:#ffa500;">#ffa500</h2>
<h2 style="background-color:#6a5acd;">#6a5acd</h2>

</body>
</html>
```



#ff0000



#0000ff



#3cb371



#ee82ee



#ffa500



#6a5acd

```
<h2 style="background-color:#ff0000;">#ff0000</h2>
<h2 style="background-color:#0000ff;">#0000ff</h2>
<h2 style="background-color:#3cb371;">#3cb371</h2>
<h2 style="background-color:#ee82ee;">#ee82ee</h2>
<h2 style="background-color:#ffa500;">#ffa500</h2>
<h2 style="background-color:#6a5acd;">#6a5acd</h2>
```

Shades of gray are often defined using equal values for all the 3 light sources:



#3c3c3c



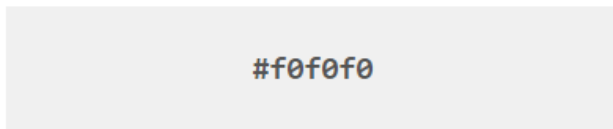
#616161



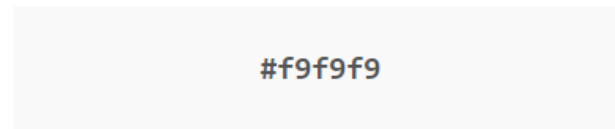
#787878



#b4b4b4



#f0f0f0



#f9f9f9

```
<h2 style="background-color:#3c3c3c;">#3c3c3c</h2>
<h2 style="background-color:#616161;">#616161</h2>
<h2 style="background-color:#787878;">#787878</h2>
<h2 style="background-color:#b4b4b4;">#b4b4b4</h2>
<h2 style="background-color:#f0f0f0;">#f0f0f0</h2>
<h2 style="background-color:#f9f9f9;">#f9f9f9</h2>
```

CSS HSL Colors

HSL stands for hue, saturation, and lightness.

HSL Value

- In CSS, a color can be specified using hue, saturation, and lightness (HSL) in the form:
- `hsl(hue, saturation, lightness)`
- Hue is a degree on the color wheel from 0 to 360. 0 is red, 120 is green, and 240 is blue.
- Saturation is a percentage value. 0% means a shade of gray, and 100% is the full color.
- Lightness is also a percentage. 0% is black, 50% is neither light or dark, 100% is white
- Experiment by mixing the HSL values below:

Example

<code>hsl(0, 100%, 50%)</code>	<code>hsl(240, 100%, 50%)</code>
<code>hsl(147, 50%, 47%)</code>	<code>hsl(300, 76%, 72%)</code>
<code>hsl(39, 100%, 50%)</code>	<code>hsl(248, 53%, 58%)</code>

```
<h2 style="background-color:hsl(0, 100%, 50%);">hsl(0, 100%, 50%)</h2>
<h2 style="background-color:hsl(240, 100%, 50%);">hsl(240, 100%, 50%)</h2>
<h2 style="background-color:hsl(147, 50%, 47%);">hsl(147, 50%, 47%)</h2>
<h2 style="background-color:hsl(300, 76%, 72%);">hsl(300, 76%, 72%)</h2>
<h2 style="background-color:hsl(39, 100%, 50%);">hsl(39, 100%, 50%)</h2>
<h2 style="background-color:hsl(248, 53%, 58%);">hsl(248, 53%, 58%)</h2>
```

❑ Saturation

- Saturation can be described as the intensity of a color.
- 100% is pure color, no shades of gray.
- 50% is 50% gray, but you can still see the color.
- 0% is completely gray; you can no longer see the color.

❑ Lightness

- The lightness of a color can be described as how much light you want to give the color, where 0% means no light (black), 50% means 50% light (neither dark nor light) and 100% means full lightness (white).

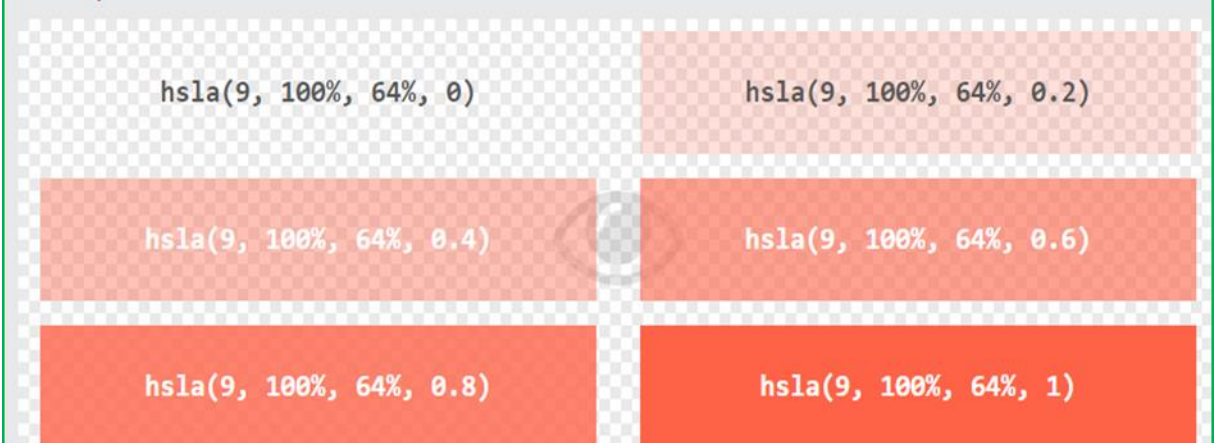
Shades of Gray

Shades of gray are often defined by setting the hue and saturation to 0, and adjust the lightness from 0% to 100% to get darker/lighter shades:

HSLA Value

- HSLA color values are an extension of HSL color values with an alpha channel - which specifies the opacity for a color.
- An HSLA color value is specified with:
- **hsla(hue, saturation, lightness, alpha)**
- The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (not transparent at all):
- Experiment by mixing the HSLA values below:

Example



```
<h2 style="background-color:hsla(9, 100%, 64%, 0);">hsla(9, 100%, 64%, 0)</h2>  
<h2 style="background-color:hsla(9, 100%, 64%, 0.2);">hsla(9, 100%, 64%, 0.2)</h2>  
<h2 style="background-color:hsla(9, 100%, 64%, 0.4);">hsla(9, 100%, 64%, 0.4)</h2>  
<h2 style="background-color:hsla(9, 100%, 64%, 0.6);">hsla(9, 100%, 64%, 0.6)</h2>
```

CSS Backgrounds

The CSS background properties are used to add background effects for elements.

In these chapters, you will learn about the following CSS background properties:

- background-color
- background-image
- background-repeat
- background-attachment
- background-position
- background (shorthand property)

❑ CSS background-color

The background-color property specifies the background color of an element.

```
<!DOCTYPE html>
<html>
<head>
<style>
h1 {
  background-color: green;
}

div {
  background-color: lightblue;
}

p {
  background-color: yellow;
}
</style>
</head>
<body>

<h1>CSS background-color example!</h1>
<div>
This is a text inside a div element.
<p>This paragraph has its own background color.</p>
We are still in the div element.
</div>

</body>
</html>
```

❑ Opacity / Transparency

- The opacity property specifies the opacity/transparency of an element.
- It can take a value from 0.0 - 1.0. The lower value, the more transparent:



```
<style>
div {
  background-color: green;
}
```

```
div.first {
  opacity: 0.1;
}
```

```
div.second {
  opacity: 0.3;
}
```

```
div.third {
  opacity: 0.6;
}
```

</style>

Transparent Boxes</h1>

<p>When using the opacity property to add transparency to the background of an element, all of its child elements become transparent as well. This can make the text inside a fully transparent element hard to read:</p>

```
<div class="first">
  <h1>opacity 0.1</h1>
</div>
```

```
<div class="second">
  <h1>opacity 0.3</h1>
</div>
```

```
<div class="third">
  <h1>opacity 0.6</h1>
</div>
```

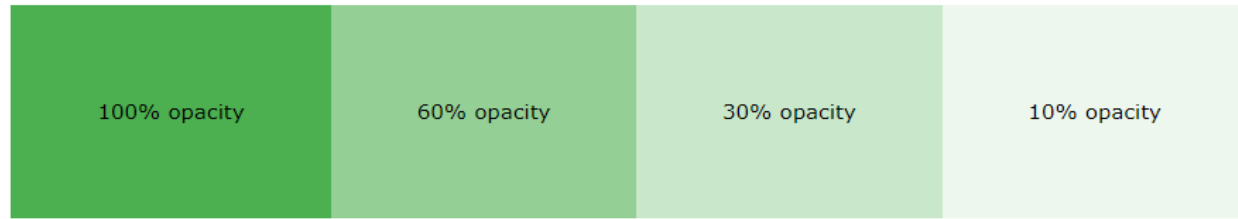
```
<div>
  <h1>opacity 1 (default)</h1>
</div>
```

```
</body>
</html>
```

Note: When using the opacity property to add transparency to the background of an element, all of its child elements inherit the same transparency. This can make the text inside a fully transparent element hard to read.

❑ Transparency using RGBA

If you do not want to apply opacity to child elements, like in our example above, use **RGBA** color values. The following example sets the opacity for the background color and not the text:



- You learned from our CSS Colors Chapter, that you can use RGB as a color value. In addition to RGB, you can use an RGB color value with an alpha channel (RGBA) - which specifies the opacity for a color.
- An RGBA color value is specified with: `rgba(red, green, blue, alpha)`. The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (fully opaque).

```
div {  
  background: rgba(0, 128, 0, 0.3) /* Green background with 30%  
opacity */  
}
```

Class Demo

❑ CSS Background Image

- The background-image property specifies an image to use as the background of an element.
- By default, the image is repeated so it covers the entire element.

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-image: url("paper.gif");
}
</style>
</head>
<body>

<h1>Hello World!</h1>

<p>This page has an image as the background!</p>

</body>
</html>
```

❑ CSS background-image Property

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-image: url("paper.gif");
  background-color: #cccccc;
}
</style>
</head>
<body>

<h1>The background-image Property</h1>

<p>Hello World!</p>

</body>
</html>
```


Example

Sets two background images for the <body> element. Let the first image appear only once (with no-repeat), and let the second image be repeated:

```
body {  
    background-image: url("img_tree.gif"),  
    url("paper.gif");  
    background-repeat: no-repeat, repeat;  
    background-color: #cccccc;
```

```
<html>
<head>
<meta name="viewport" content="width=device-width, initial-
scale=1">
<style>
body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
}
```

```
.hero-image {
  background-image: url("/w3images/photographer.jpg");
  background-color: #cccccc;
  height: 500px;
  background-position: center;
  background-repeat: no-repeat;
  background-size: cover;
  position: relative;
}
```

```
.hero-text {
  text-align: center;
  position: absolute;
  top: 50%;
  left: 50%;
```

❑ CSS background-repeat

- By default, the background-image property repeats the background image horizontally and vertically. Some images are repeated only horizontally or vertically.
- The background looks better if the image repeated horizontally only.
- Example-1

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
    background-image: url("gradient_bg.png");
    background-repeat: repeat-x;
}
</style>
</head>
<body>
<h1>Hello Students how are you </h1>
</body>
</html>
```

❑ CSS background-attachment

- The background-attachment property is used to specify if the background image is fixed or scroll with the rest of the page in browser window. If you set fixed the background image then the image will not move during scrolling in the browser. Let's take an example with fixed background image.
 - background: white url('bbb.gif');
 - background-repeat: no-repeat;
 - background-attachment: fixed;

❑ CSS background-position

The background-position property is used to define the initial position of the background image. By default, the background image is placed on the top-left of the webpage.

You can set the following positions:

- center
- top
- bottom
- left
- right

```
background: white url('good-morning.jpg');
background-repeat: no-repeat;
background-attachment: fixed;
background-position: center;
```


3.4. Colors; Backgrounds; Borders; Text; Font; List; Table

❑ CSS Border

The CSS border is a shorthand property used to set the border on an element.

The CSS border properties are use to specify the style, color and size of the border of an element. The CSS border properties are given below

- border-style
- border-color
- border-width
- border-radius

CSS border-style

The Border style property is used to specify the border type which you want to display on the web page.

There are some border style values which are used with border-style property to define a border.

Value	Description
none	It doesn't define any border.
dotted	It is used to define a dotted border.
dashed	It is used to define a dashed border.
solid	It is used to define a solid border.
double	It defines two borders wIth the same border-width value.
groove	It defines a 3d grooved border. effect is generated according to border-color value.
ridge	It defines a 3d ridged border. effect is generated according to border-color value.
inset	It defines a 3d inset border. effect is generated according to border-color value.
outset	It defines a 3d outset border. effect is generated according to border-color value.

```
<!DOCTYPE html>

<html>

<head>

<style>

p.none {border-style: none;}

p.dotted {border-style: dotted;}

p.dashed {border-style: dashed;}

p.solid {border-style: solid;}

p.double {border-style: double;}

p.groove {border-style: groove;}

p.ridge {border-style: ridge;}

p.inset {border-style: inset;}

p.outset {border-style: outset;}

p.hidden {border-style: hidden;}

</style>
```

```
</head>

<body>

<p class="none">No border.</p>

<p class="dotted">A dotted border.</p>

<p class="dashed">A dashed border.</p>

<p class="solid">A solid border.</p>

<p class="double">A double border.</p>

<p class="groove">A groove border.</p>

<p class="ridge">A ridge border.</p>

<p class="inset">An inset border.</p>

<p class="outset">An outset border.</p>

<p class="hidden">A hidden border.</p>

</body>

</html>
```

❑ CSS border-width

The border-width property is used to set the border's width. It is set in pixels. You can also use the one of the three pre-defined values, thin, medium or thick to set the width of the border.

Note: The border-width property is not used alone. It is always used with other border properties like "border-style" property to set the border first otherwise it will not work.

```
<!DOCTYPE html>
<html>
<head>
<style>
p.one {
    border-style: solid;
    border-width: 5px;
}
p.two {
    border-style: solid;
    border-width: medium;
}
p.three {
    border-style: solid;
    border-width: 1px;
```

❑ CSS border-color

There are three methods to set the color of the border.

- **Name:** It specifies the color name. For example: "red".
- **RGB:** It specifies the RGB value of the color. For example: "rgb(255,0,0)".
- **Hex:** It specifies the hex value of the color. For example: "#ff0000".
- There is also a border color named "transparent". If the border color is not set it is inherited from the color property of the element.

Note: The border-color property is not used alone. It is always used with other border properties like "border-style" property to set the border first otherwise it will not work.

```
<style>
p.one {
    border-style: solid;
    border-color: red;
}
p.two {
    border-style: solid;
    border-color: #98bf21;
}
```

❑ CSS border-radius property

- This CSS property sets the rounded borders and provides the rounded corners around an element, tags, or div. It defines the radius of the corners of an element.
- It is shorthand for border-top-left-radius, border-top-right-radius, border-bottom-right-radius and border-bottom-left-radius. It gives the rounded shape to the corners of the border of an element. We can specify the border for all four corners of the box in a single declaration using the border-radius. The values of this property can be defined in percentage or length units.

If the bottom-left value is omitted, then it will be same as the top-right. If the value of bottom-right is eliminated, then it will be same as the top-left. Similarly, if top-right is eliminated, then it will be the same as top-left.

Let's see what happens when we provide a single value, two values, three values, and four values to this property.

- If we provide a single value (such as border-radius: 30px;) to this property, it will set all corners to the same value.
- When we specify two values (such as border-radius: 20% 10% ;), then the first value will be used for the top-left and bottom-right corners, and the second value will be used for the top-right and bottom-left corners.
- When we use three values (such as border-radius: 10% 30% 20%;) then the first value will be used for the top-left corner, the second value will be applied on top-right, and bottom-left corners and the third value will be applied to the bottom-right corner.
- Similarly, when this property has four values (border-radius: 10% 30% 20% 40%;) then the first value will be the radius of top-left, the second value will be used for the top-right, the third value will be applied on bottom-right, and the fourth value is used for bottom-left.

Property	Description
border-top-left-radius	It is used to set the border-radius for the top-left corner
border-top-right-radius	It is used to set the border-radius for the top-right corner
border-bottom-right-radius	It is used to set the border-radius for the bottom-right corner
border-bottom-left-radius	It is used to set the border-radius for the bottom-left corner

Syntax

```
border-radius: 1-4 length | % / 1-4 length | % | inherit | initial;
```

Property values

length: It defines the shape of the corners. It denotes the size of the radius using length values. Its default value is 0. It does not allow negative values.

percentage: It denotes the size of the radius in percentage. It also does not allow negative values.

Example

```
<!DOCTYPE html>
<html>

<head>
<title> CSS border-radius </title>
<style>
div {
padding: 50px;
margin: 20px;
border: 6px ridge red;
width: 300px;
float: left;
height: 150px;
}
p{
font-size: 25px;
}
#one {
border-radius: 90px;
background: lightgreen;
}
#two {
border-radius: 25% 10%;
background: orange;
}
```

```
#three {
border-radius: 35px 10em
10%;
background: cyan;
}
#four {
border-radius: 50px 50%
50cm 50em;
background: lightblue;
}

</style>
```

```
</head>

<body>
<div id = "one">
<h2> Welcome to the javaTpoint.com </h2>
<p> border-radius: 90px; </p>
</div>
<div id = "two">
<h2> Welcome to Patan Multiple Campus </h2>
<p> border-radius: 25% 10%; </p>
</div>
<div id = "three">
<h2> Welcome to Madan Bhandari Memorial</h2>
<p> border-radius: 35px 10em 10%; </p>
</div>
<div id = "four">
<h2>Welcome to Xavier college </h2>
<p>border-radius: 50px 50% 50cm 50em;</p>
</div>
</body>
</html>
```

❑ CSS Margins

- The CSS margin properties are used to create space around elements, outside of any defined borders.
- With CSS, you have full control over the margins. There are properties for setting the margin for each side of an element (top, right, bottom, and left).
- CSS has properties for specifying the margin for each side of an element:
 - margin-top
 - margin-right
 - margin-bottom
 - margin-left
- All the margin properties can have the following values:
 - auto - the browser calculates the margin
 - length - specifies a margin in px, pt, cm, etc.
 - % - specifies a margin in % of the width of the containing element
 - inherit - specifies that the margin should be inherited from the parent element

Tip: Negative values are allowed.

Example

Set different margins for all four sides of a <p> element:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  border: 1px solid black;
  margin-top: 100px;
  margin-bottom: 100px;
  margin-right: 150px;
  margin-left: 80px;
  background-color: lightblue;
}
</style>
</head>
<body>
```

<h2>Using individual margin properties</h2>

<div>This div element has a top margin of 100px, a right margin of 150px, a bottom margin of 100px, and a left margin of 80px.</div>

```
</body>
</html>
```

❑ CSS Padding

- The CSS padding properties are used to generate space around an element's content, inside of any defined borders.
- With CSS, you have full control over the padding. There are properties for setting the padding for each side of an element (top, right, bottom, and left).
- Padding - Individual Sides
- CSS has properties for specifying the padding for each side of an element:
 - padding-top
 - padding-right
 - padding-bottom
 - padding-left
- All the padding properties can have the following values:
 - length - specifies a padding in px, pt, cm, etc.
 - % - specifies a padding in % of the width of the containing element
 - inherit - specifies that the padding should be inherited from the parent element
 - Note: Negative values are not allowed.

Example

Set different padding for all four sides of a <div> element:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  border: 1px solid black;
  background-color: lightblue;
  padding-top: 50px;
  padding-right: 30px;
  padding-bottom: 50px;
  padding-left: 80px;
}
</style>
</head>
<body>
```

<h2>Using individual padding properties</h2>

<div>This div element has a top padding of 50px, a right padding of 30px, a bottom padding of 50px, and a left padding of 80px.</div>

```
</body>
```

```
</html>
```

❑ CSS Height, Width and Max-width

- The CSS height and width properties are used to set the height and width of an element.
- The CSS max-width property is used to set the maximum width of an element.
- This element has a height of 50 pixels and a width of 100%.

❑ CSS Setting height and width

- The height and width properties are used to set the height and width of an element.
- The height and width properties do not include padding, borders, or margins. It sets the height/width of the area inside the padding, border, and margin of the element.

❑ CSS height and width Values

- The height and width properties may have the following values:
 - auto - This is default. The browser calculates the height and width
 - length - Defines the height/width in px, cm, etc.
 - % - Defines the height/width in percent of the containing block
 - initial - Sets the height/width to its default value
 - inherit - The height/width will be inherited from its parent value

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
    height: 50px;
    width: 100%;
    border: 1px solid #4CAF50;
}
</style>
</head>
<body>

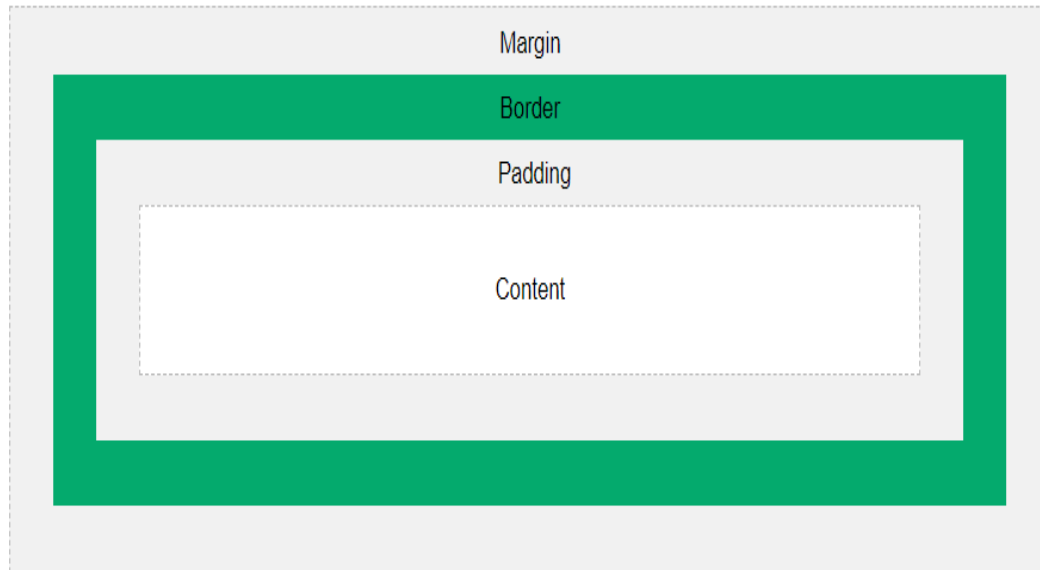
<h2>CSS height and width properties</h2>

<div>This div element has a height of 50 pixels and a width of
100%.</div>

</body>
</html>
```

❑ CSS Box Model

- In CSS, the term "box model" is used when talking about design and layout.
- The CSS box model is essentially a box that wraps around every HTML element. It consists of: margins, borders, padding, and the actual content. The image below illustrates the box model:



Explanation of the different parts:

- Content - The content of the box, where text and images appear
- Padding - Clears an area around the content. The padding is transparent
- Border - A border that goes around the padding and content
- Margin - Clears an area outside the border. The margin is transparent
- The box model allows us to add a border around elements, and to define space between elements.

Example

Demonstration of the box model:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  background-color: lightgrey;
  width: 300px;
  border: 15px solid green;
  padding: 50px;
  margin: 20px;
}
</style>
</head>
<body>

<h2>Demonstrating the Box Model</h2>

<p>The CSS box model is essentially a box that wraps around every
HTML element. It consists of: borders, padding, margins, and the actual
content.</p>

<div>This text is the content of the box. We have added a 50px padding,
20px margin and a 15px green border. Ut enim ad minim veniam, quis
nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat
non proident, sunt in culpa qui officia deserunt mollit anim id est
laborum.</div>

</body>
</html>
```

❑ Width and Height of an Element

- In order to set the width and height of an element correctly in all browsers, you need to know how the box model works.
- Important: When you set the width and height properties of an element with CSS, you just set the width and height of the content area. To calculate the full size of an element, you must also add padding, borders and margins.
- Here is the calculation:

320px (width)
+ 20px (left + right padding)
+ 10px (left + right border)
+ 0px (left + right margin)
= 350px
- The total width of an element should be calculated like this:
- Total element width = width + left padding + right padding + left border + right border + left margin + right margin
- The total height of an element should be calculated like this:
- Total element height = height + top padding + bottom padding + top border + bottom border + top margin + bottom margin

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 320px;
  padding: 10px;
  border: 5px solid gray;
  margin: 0;
}
</style>
</head>
<body>

<h2>Calculate the total width:</h2>


<div>The picture above is 350px wide. The total width of this element
is also 350px.</div>

</body>
</html>
```


❑ CSS Text

❖ Text Color

- The color property is used to set the color of the text. The color is specified by:
 - a color name - like "red"
 - a HEX value - like "#ff0000"
 - an RGB value - like "rgb(255,0,0)"
 - Look at CSS Color Values for a complete list of possible color values.
- The default text color for a page is defined in the body selector.

❖ Text Alignment and Text Direction

- text-align
- text-align-last
- direction
- unicode-bidi
- vertical-align

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: lightgrey;
  color: blue;
}

h1 {
  background-color: black;
  color: white;
}

div {
  background-color: blue;
  color: white;
}
</style>
</head>
<body>

<h1>This is a Heading</h1>
<p>This page has a grey background color and a blue text.</p>
<div>This is a div.</div>

</body>
</html>
```

The CSS Text Alignment/Direction Properties

Property	Description
direction	Specifies the text direction/writing direction
text-align	Specifies the horizontal alignment of text
text-align-last	Specifies how to align the last line of a text
unicode-bidi	Used together with the direction property to set or return whether the text should be overridden to support multiple languages in the same document
vertical-align	Sets the vertical alignment of an element

❏ Text Transformation

The text-transform property is used to specify uppercase and lowercase letters in a text.

It can be used to turn everything into uppercase or lowercase letters, or capitalize the first letter of each word:

```
p.uppercase {
  text-transform: uppercase;
}
```

```
p.lowercase {
  text-transform: lowercase;
}
```

```
p.capitalize {
  text-transform: capitalize;
```

```
<!DOCTYPE html>
<html>
<head>
<style>
p.uppercase {
  text-transform: uppercase;
}

p.lowercase {
  text-transform: lowercase;
}

p.capitalize {
  text-transform: capitalize;
}
</style>
</head>
<body>

<h1>Using the text-transform property</h1>

<p class="uppercase">This text is transformed to uppercase.</p>
<p class="lowercase">This text is transformed to lowercase.</p>
<p class="capitalize">This text is capitalized.</p>

</body>
</html>
```

CSS Fonts

- Font Selection is Important
 - Choosing the right font has a huge impact on how the readers experience a website.
 - The right font can create a strong identity for your brand.
 - Using a font that is easy to read is important. The font adds value to your text. It is also important to choose the correct color and text size for the font.
- ❑ Generic Font Families
- In CSS there are five generic font families:
 1. **Serif** fonts have a small stroke at the edges of each letter. They create a sense of formality and elegance.
 2. **Sans-serif** fonts have clean lines (no small strokes attached). They create a modern and minimalistic look.
 3. **Monospace** fonts - here all the letters have the same fixed width. They create a mechanical look.
 4. **Cursive** fonts imitate human handwriting.
 5. **Fantasy** fonts are decorative/playful fonts.
 6. All the different font names belong to one of the generic font families.

```
<!DOCTYPE html>
<html>
<head>
<style>
.p1 {
  font-family: "Times New Roman", Times, serif;
}

.p2 {
  font-family: Arial, Helvetica, sans-serif;
}

.p3 {
  font-family: "Lucida Console", "Courier New", monospace;
}
</style>
</head>
<body>

<h1>CSS font-family</h1>
<p class="p1">This is a paragraph, shown in the Times New Roman
font.</p>
<p class="p2">This is a paragraph, shown in the Arial font.</p>
<p class="p3">This is a paragraph, shown in the Lucida Console
font.</p>

</body>
</html>
```

CSS Tables Class Demo Practical

```
<!DOCTYPE html>
<html>
<head>
<style>
#customers {
font-family: Arial, Helvetica, sans-serif;
border-collapse: collapse;
width: 100%;
}

#customers td, #customers th {
border: 1px solid #ddd;
padding: 8px;
}

#customers tr:nth-child(even){background-color:
#f2f2f2;}

#customers tr:hover {background-color: #ddd;}

#customers th {
padding-top: 12px;
padding-bottom: 12px;
text-align: left;
background-color: #04AA6D;
color: white;
}
</style>
</head>
<body>

<h1>A Fancy Table</h1>
```

```
<table id="customers">
<tr>
<th>Company</th>
<th>Contact</th>
<th>Country</th>
</tr>
<tr>
<td>Alfreds Futterkiste</td>
<td>Maria Anders</td>
<td>Germany</td>
</tr>
<tr>
<td>Berglunds snabbköp</td>
<td>Christina Berglund</td>
<td>Sweden</td>
</tr>
<tr>
<td>Centro comercial Moctezuma</td>
<td>Francisco Chang</td>
<td>Mexico</td>
</tr>
<tr>
<td>Ernst Handel</td>
<td>Roland Mendel</td>
<td>Austria</td>
</tr>
```

```
<tr>
<td>Island Trading</td>
<td>Helen Bennett</td>
<td>UK</td>
</tr>
<tr>
<td>Königlich Essen</td>
<td>Philip Cramer</td>
<td>Germany</td>
</tr>
<tr>
<td>Laughing Bacchus Winecellars</td>
<td>Yoshi Tannamuri</td>
<td>Canada</td>
</tr>
<tr>
<td>Magazzini Alimentari Riuniti</td>
<td>Giovanni Rovelli</td>
<td>Italy</td>
</tr>
<tr>
<td>North/South</td>
<td>Simon Crowther</td>
<td>UK</td>
</tr>
<tr>
<td>Paris spécialités</td>
<td>Marie Bertrand</td>
<td>France</td>
</tr>
</table>

</body>
</html>
```

CSS Forms Practical Demo

```
<!DOCTYPE html>
<html>
<style>
input[type=text], select {
  width: 100%;
  padding: 12px 20px;
  margin: 8px 0;
  display: inline-block;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box;
}

input[type=submit] {
  width: 100%;
  background-color: #4CAF50;
  color: white;
  padding: 14px 20px;
  margin: 8px 0;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

input[type=submit]:hover {
  background-color: #45a049;
}

div {
  border-radius: 5px;
  background-color: #f2f2f2;
  padding: 20px;
}
</style>
```

```
<body>

<h3>Using CSS to style an HTML Form</h3>

<div>
  <form action="/action_page.php">
    <label for="fname">First Name</label>
    <input type="text" id="fname" name="firstname" placeholder="Your name..">

    <label for="lname">Last Name</label>
    <input type="text" id="lname" name="lastname" placeholder="Your last
name..">

    <label for="country">Country</label>
    <select id="country" name="country">
      <option value="australia">Australia</option>
      <option value="canada">Canada</option>
      <option value="usa">USA</option>
    </select>

    <input type="submit" value="Submit">
  </form>
</div>

</body>
</html>
```

Example 2: This example illustrates the Box Model by implementing the various properties.

```
<!DOCTYPE html>
<head>
  <style>
    .main {
      font-size: 32px;
      font-weight: bold;
      text-align: center;
    }
    #box {
      padding-top: 40px;
      width: 400px;
      height: 100px;
      border: 50px solid green;
      margin: 50px;
      text-align: center;
      font-size: 32px;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <div class="main">CSS Box-Model Property</div>
  <div id="box">Web Technology</div>
</body>
</html>
```

Example 1: This example illustrates the use of the CSS Box model for aligning & displaying it properly.

```
<!DOCTYPE html>
<head>
  <title>CSS Box Model</title>
  <style>
    .main {
      font-size: 36px;
      font-weight: bold;
      Text-align: center;
    }
    .gfg {
      margin-left: 60px;
      border: 50px solid #009900;
      width: 300px;
      height: 200px;
      text-align: center;
      padding: 50px;
    }
    .gfg1 {
      font-size: 42px;
      font-weight: bold;
      color: #009900;
      margin-top: 60px;
      background-color: #c5c5db;
    }
  </style>
</head>
<body>
  <div class="main">CSS
  Box-Model Property</div>
  <div class="gfg">
    <div
    class="gfg1">Web Technology</div>
    <div
    class="gfg2">
      World cup 2022
    </div>
  </div>
</body>
</html>
```

```
.gfg2 {
  font-size: 18px;
  font-weight: bold;
  background-color: #c5c5db;
}
</style>
</head>
<body>
  <div class="main">CSS
  Box-Model Property</div>
  <div class="gfg">
    <div
    class="gfg1">Web Technology</div>
    <div
    class="gfg2">
      World cup 2022
    </div>
  </div>
</body>
</html>
```

3.8. Basics of Responsive Web Designs; Media Queries (Media Types, Viewport)

What is Media Query?

- CSS Media query is a W3C recommendation and a CSS3 module which is used to adapt to conditions such as screen resolution (e.g. Smartphone screen vs. computer screen).
- The media query technique first used in CSS3.
- It became a W3C recommendation in June 2012.
- It is an extension of media dependent stylesheets used in different media types (i.e. screen and print) found in CSS2.
- The most commonly used media feature is "width".
- It uses the @media rule to include a block of CSS properties only if a certain condition is true.

What is Responsive Web Design?

- The term Responsive Web Design was given by Ethan Marcotte.
- It facilitates you to use fluid grids, flexible images, and media queries to progressively enhance a web page for different viewing contexts i.e. Desktop, Smartphone, Tablet etc.

Smartphone: 320px

Tablet: 768px

Netbook: 1024px

Desktop: 1600px

What is the output

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-
width, initial-scale=1">
<style>
div.example {
  background-color: yellow;
  padding: 20px;
}

@media screen and (max-width: 600px) {
  div.example {
    display: none;
  }
}
```

```
</style>
</head>
<body>

<h2>Hide elements on different screen
sizes</h2>

<div class="example">Example DIV.</div>

<p>When the browser's width is 600px wide or
less, hide the div element. Resize the browser
window to see the effect.</p>

</body>
</html>
```


Change Font Size With Media Queries

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-
width, initial-scale=1">
<style>
div.example {
  background-color: lightgrey;
  padding: 20px;
}

@media screen and (min-width: 600px) {
  div.example {
    font-size: 80px;
  }
}

@media screen and (max-width: 600px) {
  div.example {
    font-size: 30px;
  }
}
```

```
</style>
</head>
<body>

<h2>Change the font size of an element on
different screen sizes</h2>

<div class="example">Example DIV.</div>

<p>When the browser's width is 600px wide or
less, set the font-size of DIV to 30px. When it is
601px or wider, set the font-size to 80px. Resize
the browser window to see the effect.</p>

</body>
</html>
```

Orientation: Portrait / Landscape

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<style>
body {
  background-color: lightgreen;
}

@media only screen and (orientation: landscape) {
  body {
    background-color: lightblue;
  }
}
</style>
</head>
<body>

<p>Resize the browser window. When the width of this document is larger than
the height, the background color is "lightblue", otherwise it is "lightgreen".</p>

</body>
</html>
```

Min Width to Max Width

- You can also use the (max-width: ..) and (min-width: ..) values to set a minimum width and a maximum width.
- For example, when the browser's width is between 600 and 900px, change the appearance of a <div> element:

Example

```
@media screen and (max-width: 900px) and (min-width:
600px) {
  div.example {
    font-size: 50px;
    padding: 50px;
    border: 8px solid black;
    background: yellow;
  }
}
```

Example: Min Width to Max Width

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
@media screen and (max-width: 900px) and (min-width: 600px) {
  div.example {
    font-size: 50px;
    padding: 50px;
    border: 8px solid black;
    background: yellow;
  }
}
</style>
</head>
<body>

<h2>Change the appearance of DIV on different screen sizes</h2>

<div class="example">Example DIV.</div>

<p>When the browser's width is between 600 and 900px, change the appearance
of DIV.
<strong>Resize the browser window to see the effect</strong>.</p>

</body>
</html>
```

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
@media screen and (max-width: 900px) and (min-width: 600px), (min-
width: 1100px) {
  div.example{
    font-size: 50px;
    padding: 50px;
    border: 8px solid black;
    background: yellow;
  }
}
</style>
</head>
<body>

<h2>Change the appearance of DIV on different screen sizes</h2>

<div class="example">Example DIV.</div>

<p>When the browser's width is between 600 and 900px OR above 1100px,
change the appearance of DIV.
<strong>Resize the browser window to see the effect</strong>.</p>

</body>
</html>
```

Example-1

This example specifies that if you resize your window less than 500px, the background color will be changed.

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
<style>
body {
    background-color:yellow;
}

@media (max-width: 500px) {
    body {
        background-color:green;
    }
}
</style>
</head>
<body>
<p>If you resize the browser window and the width of this
document is less than 500 pixels, the background-color is "green",
otherwise it is "yellow"</p>
</body>
</html>
```

Example-2

body>

```
<h1>Resize the browser window to see the
effect!</h1>
<p>By default, the background color of the
document is "tan". If the screen size is
992px or less, the color will change to
"blue". If it is 600px or less, it will change
to "olive".</p>
```

```
</body>
</html>
```

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
    background-color: tan;
    color: black;
}

/* On screens that are 992px wide or less,
the background color is blue */
@media screen and (max-width: 992px) {
    body {
        background-color: blue;
        color: white;
    }
}

/* On screens that are 600px wide or less,
the background color is olive */
@media screen and (max-width: 600px) {
    body {
        background-color: olive;
        color: white;
    }
}
</style>
</head>
```

Media Queries For Menus

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport"
content="width=device-width, initial-scale=1">
<style>
* {
  box-sizing: border-box;
}

/* Style the top navigation bar */
.topnav {
  overflow: hidden;
  background-color: #333;
}
* Style the topnav links */
.topnav a {
  float: left;
  display: block;
  color: #f2f2f2;
  text-align: center;
  padding: 14px 16px;
  text-decoration: none;
}
```

```
/* Change color on hover */
.topnav a:hover {
  background-color: #ddd;
  color: black;
}

/* On screens that are 600px wide or less, make the
menu links stack on top of each other instead of next
to each other */
@media screen and (max-width: 600px) {
  .topnav a {
    float: none;
    width: 100%;
  }
}
</style>
```

```
</head>
<body>

<h2>Responsive navigation
menu</h2>
<p>Resize the browser window to see
the effect: When the screen is less than
600px, the navigation menu will be
displayed vertically instead of
horizontally.</p>

<div class="topnav">
  <a href="#">Link</a>
  <a href="#">Link</a>
  <a href="#">Link</a>
</div>

</body>
</html>
```

```

<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width,
initial-scale=1">
<style>
* {
box-sizing: border-box;
}

/* Create four equal columns that floats next to each
other */
column {
float: left;
width: 25%;
padding: 20px;
}

/* Clear floats after the columns */
row:after {
content: "";
display: table;
clear: both;
}

```

```

/* On screens that are 992px wide or
less, go from four columns to two
columns */
@media screen and (max-width:
992px) {
.column {
width: 50%;
}
}

/* On screens that are 600px wide or
less, make the columns stack on top
of each other instead of next to each
other */
@media screen and (max-width:
600px) {
.column {
width: 100%;
}
}
</style>
</head>

```

```
<body>
```

```
<h2>Responsive Four Column Layout</h2>
```

```

<p><strong>Resize the browser window to see the
responsive effect.</strong> On screens that are 992px
wide or less, the columns will resize from four columns
to two columns. On screens that are 600px wide or less,
the columns will stack on top of each other instead of
next to each other.</p>

```

```
<div class="row">
```

```

<div class="column" style="background-color:#aaa;">

```

```

<h2>Column 1</h2>

```

```

<p>Some text..</p>

```

```
</div>
```

```

<div class="column" style="background-color:#bbb;">

```

```

<h2>Column 2</h2>

```

```

<p>Some text..</p>

```

```
</div>
```

```

<div class="column" style="background-color:#ccc;">

```

```

<h2>Column 3</h2>

```

```

<p>Some text..</p>

```

```
</div>
```

```

<div class="column" style="background-color:#ddd;">

```

```

<h2>Column 4</h2>

```

```

<p>Some text..</p>

```

```
</div>
```